

PARTIE II : ENVIRONNEMENT TECHNIQUE

I. Choix du Framework et du Langage

1.1 PHP 8.4 : La performance brute

Pour ce projet, j'ai fait le choix de la toute dernière version de **PHP (8.4)**. Ce choix permet de bénéficier des avancées majeures en termes de typage et de performance (moteur JIT). Cela garantit une base de code propre, sécurisée et extrêmement rapide, indispensable pour une expérience utilisateur fluide.

1.2 Symfony 8 : Le standard de l'exigence

L'application repose sur le framework **Symfony 8**. J'ai sélectionné cet outil pour sa robustesse et son architecture modulaire. Symfony m'offre un cadre de développement professionnel ("Enterprise-grade") qui facilite la maintenance à long terme et assure une sécurité native contre les failles courantes (injection SQL, CSRF, XSS).

2. Infrastructure et Conteneurisation

2.1 Docker : Un environnement reproductible

Le développement a été entièrement réalisé sous **Docker**. Cette technologie permet d'isoler chaque service (PHP, Base de données, pgAdmin) dans des conteneurs étanches. Cela garantit que l'application fonctionnera de manière strictement identique sur mon poste de travail, en télétravail ou sur un serveur de production.

2.2 FrankenPHP : La modernité au service du "Zen"

Au cœur de mon infrastructure Docker, j'utilise **FrankenPHP**. C'est un serveur d'application moderne écrit en Go, qui intègre nativement les capacités de PHP. Il permet des temps de réponse ultra-courts, ce qui est crucial pour respecter le calme et la fluidité de la navigation sur le site.

3. Gestion des données : PostgreSQL 14

Pour le stockage, mon choix s'est porté sur **PostgreSQL 14**. Contrairement à MySQL, PostgreSQL offre une gestion plus fine des relations complexes et une intégrité des données supérieure. C'est un choix stratégique pour gérer avec précision les flux monétaires du Wallet et la structure riche du Lexique.

4. Front-end : Tailwind CSS v4

Pour l'interface, j'utilise la version **v4 de Tailwind CSS**. Grâce à son nouveau moteur de compilation en Rust, Tailwind v4 permet de générer un design "poids plume". Cela me permet de traduire fidèlement les maquettes Figma "Haute Couture" en conservant une rapidité de chargement exceptionnelle.

1.1 Le conseil "Pro" pour votre oral :

Si on vous demande : *"Pourquoi avoir pris des versions aussi récentes (PHP 8.4, Symfony 8) alors qu'elles viennent de sortir ?"*

Réponse : *"C'est un choix de pérennité. En commençant un projet sur les versions les plus récentes, je repousse la dette technique et je garantis à Sandrine Tribhou une plateforme qui restera moderne et sécurisée pendant plusieurs années sans nécessiter de mise à jour majeure immédiate."*

1.2 1. Le Moteur JIT (Just-In-Time)

- **C'est quoi ?** Littéralement "Compilation juste à temps".
 - **L'explication simple :** Habituellement, PHP traduit le code ligne par ligne à chaque fois qu'on appelle une page. Le moteur JIT repère les parties de code qui reviennent souvent et les transforme une fois pour toutes en "langage machine" (le langage que le processeur comprend directement).
 - **L'intérêt pour L'EMPREINTE :** "Cela booste les performances sur les calculs lourds. Pour l'utilisateur, c'est une navigation plus fluide, car le serveur répond beaucoup plus vite."
-

2. Injection SQL

- **C'est quoi ?** Une attaque où un pirate tape du code de base de données dans un formulaire (ex: dans le champ 'Nom') pour voler ou effacer des données.

- **L'explication simple :** C'est comme si quelqu'un ajoutait une commande malveillante à la fin d'un bon de commande pour forcer le commerçant à lui donner tout son stock.
 - **La protection Symfony :** "J'utilise l'ORM Doctrine. Il utilise des 'requêtes préparées' : il sépare totalement la donnée du patient de la commande SQL. Le code malveillant est traité comme du simple texte et ne peut jamais s'exécuter."
-

3. CSRF (Cross-Site Request Forgery)

- **C'est quoi ?** Une attaque qui force un utilisateur connecté à exécuter une action à son insu (ex: forger un lien qui, si Sandrine clique dessus, supprime un soin du catalogue).
 - **L'explication simple :** C'est comme si un pirate signait un chèque à votre place parce que vous avez laissé votre carnet de chèques ouvert sur la table.
 - **La protection Symfony :** "Symfony génère un **Token CSRF** (un jeton unique et secret) caché dans chaque formulaire. Quand Sandrine valide une action, Symfony vérifie que le jeton est le bon. Si un pirate tente de forger la demande, le jeton manquera et l'action sera bloquée."
-

4. XSS (Cross-Site Scripting)

- **C'est quoi ?** L'injection d'un script malveillant (JavaScript) dans une page vue par d'autres utilisateurs (ex: un pirate écrit un message dans le Lexique qui contient un script pour voler les cookies des visiteurs).
 - **L'explication simple :** C'est comme si quelqu'un collait un post-it empoisonné sur le mur du cabinet : tous ceux qui le lisent sont infectés.
 - **La protection Symfony :** "J'utilise le moteur de template **Twig**. Par défaut, Twig 'échappe' tout le contenu. Si un pirate tape `<script>`, Twig va l'afficher tel quel à l'écran sous forme de texte in-offensif au lieu de l'exécuter comme un programme."
-

Petit conseil "Zen" pour l'oral :

Si on vous demande pourquoi vous parlez de tout ça, répondez :

La sécurité fait partie de l'ADN Haute Couture du projet. On ne peut pas offrir de sérénité thérapeutique au patient si ses données personnelles et les transactions financières du cabinet ne sont pas protégées par les meilleurs standards actuels."